Arquitectura

Decisiones Técnicas y Trade-offs

DistributedProcessing, Documento 02-arquitectura

1. Resumen ejecutivo

Este documento describe las decisiones arquitectónicas principales del proyecto DistributedProcessing y los trade-offs evaluados en cada elección. El proyecto implementa un motor de procesamiento distribuido MapReduce, desarrollado desde cero en Go y comunicado mediante gRPC, desplegado sobre un clúster Kubernetes que orquesta el ciclo de vida de los contenedores. La separación de responsabilidades es explícita en todo el documento: Kubernetes resuelve la orquestación de procesos, mientras que el motor propio resuelve la lógica de particionamiento, comunicación, shuffle y tolerancia a fallos a nivel de aplicación.

2. Por qué Go

2.1. Decisión

Go fue seleccionado como el lenguaje principal para implementar el sistema de procesamiento distribuido.

2.2. Justificación

Go proporciona goroutines, threads ligeros manejados por el runtime, capaces de ejecutar miles o millones de instancias simultáneamente sin el overhead de los threads del sistema operativo, lo que lo hace ideal para sistemas distribuidos que manejan múltiples conexiones concurrentes.

El lenguaje compila a un único binario estático sin dependencias externas, lo que simplifica la distribución a múltiples nodos y encaja de forma natural con la contenedorización, reduciendo el tamaño de las imágenes generadas y el tiempo de arranque de los pods en Kubernetes.

En cuanto a rendimiento, Go es casi tan rápido como C o C++ pero considerablemente más fácil de mantener, con bajo consumo de memoria y un recolector de basura optimizado para baja latencia, lo cual resulta apropiado para sistemas con recursos limitados.

Finalmente, Go cuenta con un ecosistema robusto para sistemas distribuidos: soporte nativo y de primera clase para gRPC, librerías de red maduras como net, net/http y context, y herramientas de testing integradas para código concurrente.

2.3. Trade-offs

Aspecto	Ventaja	Desventaja
Curva de aprendizaje	Sintaxis simple, fácil de aprender	Si el equipo viene de OOP, requiere cambio de paradigma

Aspecto	Ventaja	Desventaja
Ecosistema	Grande para sistemas distribuidos	Menos librerías que Python para análisis de datos
Tipado	Tipado estático, mejor seguridad	Menos flexible que lenguajes dinámicos

2.4. Alternativas evaluadas

Se evaluó Python, descartado por ser más lento y por el overhead del GIL para concurrencia real, aunque ofrece librerías superiores para análisis de datos. Se evaluó Java, descartado por el peso de la JVM y su mayor consumo de memoria, además de imágenes de contenedor más pesadas. Se evaluó Rust, que ofrece mayor seguridad pero con una curva de aprendizaje más pronunciada que Go.

3. Por qué gRPC

3.1. Decisión

gRPC fue seleccionado como el protocolo RPC para la comunicación entre servicios distribuidos.

3.2. Justificación

gRPC utiliza Protocol Buffers para una serialización binaria compacta y rápida, y se apoya en HTTP/2 con multiplexing nativo, permitiendo múltiples streams en una sola conexión y reduciendo significativamente el ancho de banda frente a REST sobre JSON.

En cuanto a rendimiento, gRPC ofrece latencia baja y throughput alto para comunicación de alta frecuencia entre servicios, siendo típicamente uno o dos órdenes de magnitud más rápido que REST para carga de datos. Esto es especialmente relevante en el shuffle de la Etapa 2 del pipeline, donde el volumen de tráfico entre workers es mayor que en la agregación simple de la Etapa 1.

Los archivos proto definen contratos de interfaz explícitos con tipado fuerte, lo que permite validación automática de mensajes y elimina la serialización manual mediante generación de código. Además, gRPC soporta streaming bidireccional de forma nativa, habilitando patrones de mensajería más complejos entre cliente y servidor.

3.3. Trade-offs

Aspecto	Ventaja	Desventaja
Curva de aprendizaje	Potente y eficiente	Requiere aprender Protocol Buffers
Debugging	Tipado fuerte, errores claros	Mensajes binarios, no legibles en texto plano
Browser	No requiere HTTP/1.1	No soporta llamadas directas desde navegador sin gRPC-Web

Aspecto	Ventaja	Desventaja
Ecosistema REST	Herramientas especializadas	Menos omnipresente que REST en todos los lenguajes

3.4. Alternativas evaluadas

Se evaluó REST sobre JSON, más simple pero menos eficiente para comunicación de alta frecuencia. Se evaluó GraphQL, más flexible pero con overhead de parsing. Se evaluaron colas de mensajes como RabbitMQ o Kafka, adecuadas para asincronía pero más complejas para RPC sincrónico como el que requiere la asignación de tareas y los heartbeats del motor propio.

4. Por qué Kubernetes

4.1. Decisión

Kubernetes fue seleccionado como la plataforma de orquestación de contenedores para el clúster, reemplazando la gestión manual de procesos en las cuatro máquinas físicas.

4.2. Justificación

Kubernetes resuelve el problema de virtualización y orquestación de procesos: ciclo de vida de contenedores, red entre nodos, ubicación de los pods y recuperación automática de procesos caídos. Esta responsabilidad se mantiene deliberadamente separada de la lógica de distribución de datos y cómputo del motor propio, que cubre particionamiento, comunicación, shuffle, join distribuido y tolerancia a fallos a nivel de tarea.

Los workers se despliegan mediante un StatefulSet con un Service headless asociado, lo que otorga a cada worker un nombre DNS estable dentro del clúster, necesario para que el master pueda dirigirse a cada worker de forma predecible sin depender de descubrimiento dinámico. El master se despliega mediante un Deployment de una réplica con su propio Service para exponer su API gRPC.

Para garantizar que las mediciones de speedup no queden contaminadas por dos workers ejecutándose en el mismo nodo físico, se configura podAntiAffinity de forma que cada pod worker sea programado en un nodo distinto. Kubernetes gestiona además la recuperación de procesos mediante livenessProbe y readinessProbe: si un contenedor worker deja de responder, Kubernetes lo reinicia automáticamente, de forma independiente y en un plano distinto al mecanismo de reasignación de tareas que implementa el master.

Se evita deliberadamente delegar en primitivas nativas de Kubernetes, como un Job con completions indexados, la responsabilidad de repartir el trabajo entre los workers, ya que esa lógica de particionamiento y scheduling de tareas es precisamente el objeto de aprendizaje e implementación del proyecto.

4.3. Trade-offs

Aspecto	Ventaja	Desventaja
Recuperación de procesos	Automática mediante probes, sin código adicional	Opera en un plano distinto al de recuperación de tareas del motor propio, requiere medirse por separado

Aspecto	Ventaja	Desventaja
Descubrimiento de servicios	DNS estable vía Service headless	Requiere StatefulSet en lugar de Deployment simple para los workers
Curva de aprendizaje	Estándar de la industria, documentación amplia	Conceptos adicionales sobre Docker puro: pods, servicios, afinidad
Aislamiento entre nodos físicos	podAntiAffinity garantiza un worker por nodo	Requiere configuración explícita, no es el comportamiento por defecto

4.4. Alternativas evaluadas

Se evaluó gestionar los contenedores Docker de forma manual en cada una de las cuatro máquinas, descartado por no ofrecer recuperación automática de procesos ni descubrimiento de servicios, y por requerir scripts propios de despliegue difíciles de mantener a medida que el número de experimentos crece. Se evaluó Docker Swarm, más simple que Kubernetes pero con un ecosistema y una comunidad considerablemente menores, y con primitivas menos maduras para afinidad de pods. Se evaluó Nomad, viable pero con menor adopción y documentación que Kubernetes para este tipo de despliegue académico.

5. Por qué Docker como base de contenedorización

5.1. Decisión

Docker fue seleccionado como el runtime de contenedores sobre el cual corre Kubernetes, y como la herramienta de construcción de las imágenes de master y workers.

5.2. Justificación

Docker garantiza portabilidad: la misma imagen corre de forma idéntica en desarrollo y en el clúster, encapsulando completamente las dependencias del binario Go. Cada contenedor corre en su propio namespace de procesos, red y sistema de archivos, ofreciendo aislamiento de recursos y un entorno reproducible y versionable, sobre el cual Kubernetes programa y gestiona los pods.

La combinación de Go y Docker resulta particularmente favorable, ya que Go compila binarios pequeños, típicamente entre 5 y 50 MB, lo que permite imágenes muy ligeras partiendo de scratch y sin necesidad de un runtime adicional como la JVM o un intérprete de Python, reduciendo el tiempo de despliegue de los pods en el clúster.

5.3. Ejemplo: Dockerfile óptima para Go

```
FROM golang:1.26.1 AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o main .

FROM scratch
```

```
COPY --from=builder /app/main /main
EXPOSE 50051
CMD ["/main"]
```

El resultado es una imagen de aproximadamente 10 MB, sin dependencias adicionales.

5.4. Trade-offs

Aspecto	Ventaja	Desventaja
Overhead	Mínimo en Linux nativo	Overhead en Mac/Windows con Docker Desktop durante desarrollo local
Tamaño de imagen	Binario Go compila a imagen mínima	Requiere multi-stage build para evitar incluir el toolchain de Go
Debugging	Aislamiento seguro	Más complejo depurar dentro de un contenedor sin herramientas de shell
Persistencia	Volúmenes bien soportados junto con PersistentVolume de Kubernetes	Requiere gestión explícita de estado entre reinicios de pod

5.5. Alternativas evaluadas

Se evaluaron máquinas virtuales tradicionales, con más aislamiento pero un overhead masivo en gigabytes frente a megabytes, y arranque considerablemente más lento para los experimentos repetidos que requiere el proyecto. Se evaluó despliegue en bare metal, con máximo rendimiento pero incompatible con la orquestación mediante Kubernetes.

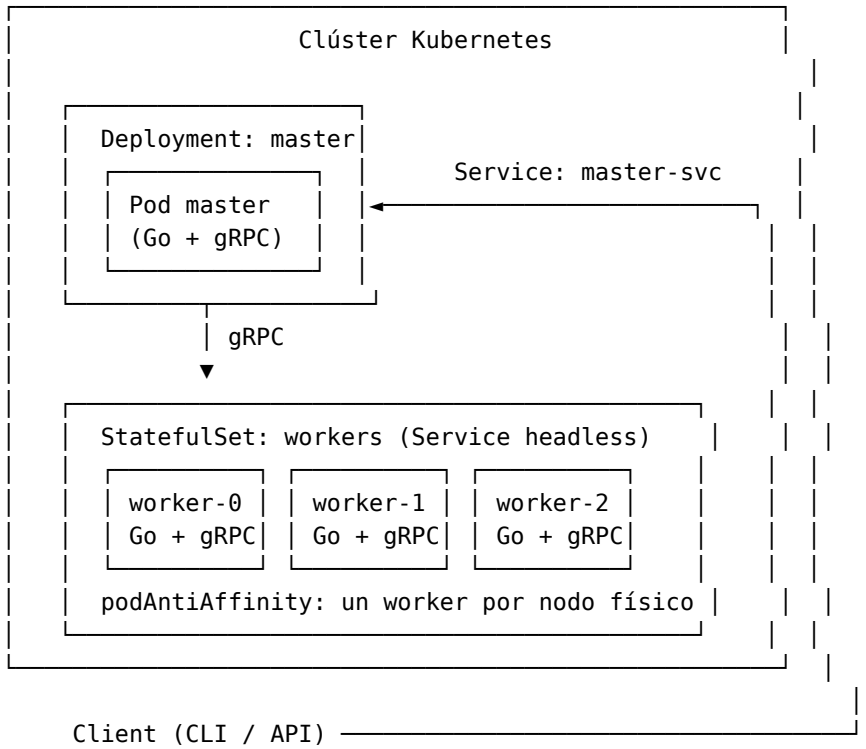
6. Matriz de decisiones: evaluación de alternativas de lenguaje

Criterio	Go	Java	Python	Rust
Concurrencia	5/5	4/5	2/5	5/5
Performance	5/5	4/5	3/5	5/5
Tamaño binario	5/5	2/5	2/5	5/5
Curva de aprendizaje	5/5	3/5	5/5	3/5
Ecosistema distribuido	5/5	4/5	3/5	3/5

7. Arquitectura general del sistema

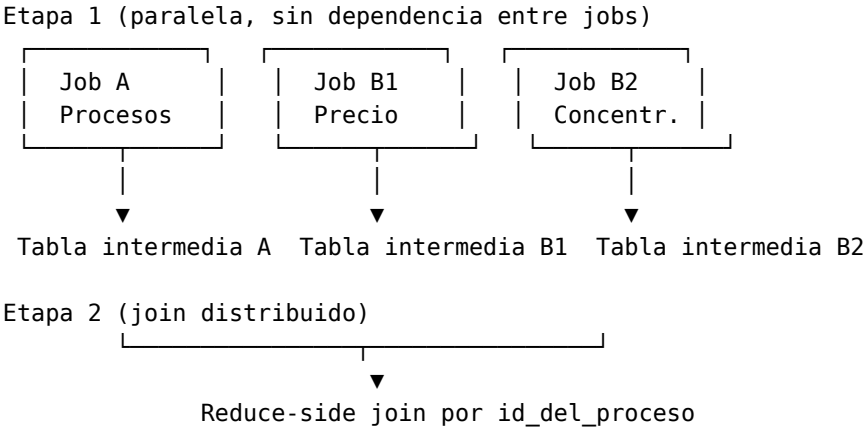
7.1. Despliegue sobre Kubernetes

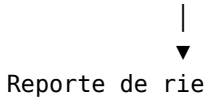
El clúster consta de un Deployment de una réplica para el master, expuesto mediante un Service, y un StatefulSet de tres réplicas para los workers, expuesto mediante un Service headless que otorga nombre DNS estable a cada pod. La configuración de podAntiAffinity asegura que cada worker se programe en un nodo físico distinto de los otros dos.



7.2. Flujo del pipeline de dos etapas

El master coordina dos etapas de procesamiento sobre los workers, según lo definido en el esquema de datos del proyecto. En la Etapa 1, los Jobs A, B1 y B2 se ejecutan de forma independiente, cada uno particionando su conjunto de datos correspondiente entre los tres workers. En la Etapa 2, el master coordina el reduce-side join sobre los resultados agregados de la Etapa 1.





7.3. Estrategia de particionamiento y asignación de datos (Chunker y Work Queue)

7.3.1. Decisión

Se adopta una estrategia de partición en micro-chunks dinámicos ($N \gg 3$ workers), donde el dataset de entrada se fragmenta en N bloques pequeños gestionados a través de una cola de trabajo (Work Queue) en memoria en el Master, descartando la partición estática 1:1 ($N = 3$).

7.3.2. Justificación y cumplimiento de métricas

La partición estática $1:1$ asignaría un tercio ($\approx 33\%$) del dataset a cada worker de manera fija. Dicho enfoque presenta dos fallas críticas:

1. **Vulnerabilidad ante trabajadores rezagados (stragglers)**: Si un worker experimenta contención de CPU o datos más costosos de procesar, todo el pipeline queda detenido esperando a dicho worker.
2. **Incompatibilidad con el SLA de recuperación de tarea**: El documento de alcance (`01-alcance.typ`) exige un tiempo máximo de recuperación de tarea ≤ 15 segundos. Si un worker falla al procesar un chunk del 33 % del dataset, reasignar y recomputar dicho bloque tomaría varios minutos, violando el umbral.

Con micro-chunks de tamaño objetivo entre 10 MB y 30 MB ($\approx$ 50.000 a 150.000 registros), el tiempo de procesamiento de cada chunk en Go es del orden de 1 a 3 segundos. Cuando el Master detecta la caída de un worker mediante el timeout de heartbeat (≤ 10 s), reasigna únicamente el micro-chunk que estaba en progreso a otro worker disponible (≤ 5 s), logrando la recuperación completa dentro del margen estricto de 15 segundos.

Adicionalmente, el esquema de Work Queue permite que los workers más rápidos procesen más bloques sin requerir balanceo manual, y reduce el consumo pico de memoria RAM en los nodos al no requerir la carga del dataset completo en un único paso.

7.3.3. Trade-offs

Aspecto	**Ventaja**	**Desventaja**
Granularidad de fallo	Pérdida mínima de trabajo ante caída (1-3s)	Mayor cantidad de mensajes RPC de asignación y reporte
Balanceo de carga	Dinámico por demanda según velocidad real de cada worker	Requiere mantener estado de cola de tareas en el Master
Consumo de memoria	Bajo y predecible por worker (procesamiento por lotes)	Overhead de deserialización gRPC distribuido en múltiples mensajes

8. Conclusiones y próximos pasos

Las decisiones de usar Go, gRPC, Docker y Kubernetes quedan ratificadas: Go resulta ideal para sistemas distribuidos con alta concurrencia, gRPC ofrece máxima eficiencia en la comunicación entre servicios,

Docker garantiza portabilidad de las imágenes, y Kubernetes resuelve la orquestación de procesos y recuperación automática de pods de forma separada y complementaria a la tolerancia a fallos a nivel de tarea que implementa el motor propio.

Como consideraciones futuras se contempla incorporar observabilidad mediante distributed tracing con Jaeger y métricas con Prometheus, implementar patrones de circuit breaker y retry logic en gRPC, y automatizar el despliegue de los manifiestos de Kubernetes mediante un pipeline de integración continua.

9. Referencias

Documentación oficial de Go en golang.org/doc, guía de gRPC en Go en grpc.io/docs/languages/go, especificación de Protocol Buffers en developers.google.com/protocol-buffers, buenas prácticas de Docker en docs.docker.com/develop/dev-best-practices y documentación de Kubernetes en kubernetes.io/docs, particularmente las secciones de StatefulSet, Service headless y podAntiAffinity.